UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA
Facultad de Ingeniería de Producción y Servicios
Escuela Profesional de Ingeniería de Sistemas

Sistemas Operativos

Semestre VI

PRÁCTICA SEMANA 6

Concurrencia y threads

Condición de carrera y reparación controlada — Semana 6

PRÁCTICA SEMANA 6

Intercalados, atomicidad y visibilidad
con threads y exclusión mutua

AUTOR
Gustavo Alonso Carrillo Villalta

DOCENTE
Norman Patrick Harvey Arce

PERIODO ACADÉMICO
2026-B

Arequipa – Perú
2026

1. Introducción

Esta práctica reproduce de forma controlada una condición de carrera y luego la corrige. Varios `threading.Thread` actualizan un contador compartido y transfieren unidades entre dos saldos compartidos, con pausas aleatorias entre la lectura y la escritura para ampliar la ventana de intercalado. La campaña se repite 100 veces por escenario y se registran las ejecuciones que violan la suma total o los límites.

En la segunda parte se definen los invariantes, se protege la transición completa con un `threading.Lock`, se repiten las mismas pruebas, se mide el costo de la protección y se deja una prueba automatizada que falla si un invariante se rompe.

Todo el código fue ejecutado y las cifras de este informe provienen de esas ejecuciones (`resultados/corridas.csv` y `resultados/resumen.txt` en el repositorio entregado).

2. Parte A — Provocar el fallo

2.1. Estado compartido y operaciones

Dato	Lectores / escritores	Operación compuesta
`contador`	todos los threads	leer → pausa → escribir `v+1`
`a`, `b`	todos los threads	verificar `saldo >= u` → pausa → debitar origen → pausa → leer destino → pausa → acreditar

Cada *paso* de un thread incrementa el contador y transfiere entre 1 y 60 unidades en dirección aleatoria entre `a` y `b`. Los saldos iniciales son 100 y 100, de modo que los límites sí pueden alcanzarse. Cada thread ejecuta 50 pasos.

2.2. Implementación

La versión sin protección y la protegida comparten el mismo código; la única diferencia es si `paso()` se ejecuta dentro de un `Lock` (parámetro `protegida`). El verificador `verificar()` comprueba tres condiciones al final de cada corrida.

Listing 1: carrera.py

```python
import random
import threading
import time
from contextlib import nullcontext

SALDO_INICIAL = 100
UNIDAD_MAX = 60


class Cuenta:
    def __init__(self, protegida=False, pausa_max=0.001, pausa=None, traza=None):
        self.contador = 0
```

```
        self.a = SALDO_INICIAL
        self.b = SALDO_INICIAL
        self.total = 2 * SALDO_INICIAL
        self.limite_roto = False
        self.protegida = protegida
        self.lock = threading.Lock() if protegida else nullcontext()
        self.pausa = pausa or (lambda: self._pausa_aleatoria(pausa_max))
        self.traza = traza

    @staticmethod
    def _pausa_aleatoria(maximo):
        if maximo > 0:
            time.sleep(random.uniform(0, maximo))

    def _log(self, evento, dato):
        if self.traza is not None:
            self.traza.append((time.perf_counter(), threading.current_thread().name, evento, dato))

    def incrementar(self):
        v = self.contador
        self._log("lee contador", v)
        self.pausa()
        self.contador = v + 1
        self._log("escribe contador", v + 1)

    def transferir(self, origen, destino, unidades):
        saldo_origen = getattr(self, origen)
        self._log("lee " + origen, saldo_origen)
        if saldo_origen < unidades:
            return False
        self.pausa()
        nuevo = getattr(self, origen) - unidades
        setattr(self, origen, nuevo)
        self._log("escribe " + origen, nuevo)
        if nuevo < 0:
            self.limite_roto = True
        saldo_destino = getattr(self, destino)
        self._log("lee " + destino, saldo_destino)
        self.pausa()
        setattr(self, destino, saldo_destino + unidades)
        self._log("escribe " + destino, saldo_destino + unidades)
        return True

    def paso(self, origen, destino, unidades):
        with self.lock:
            self.incrementar()
            self.transferir(origen, destino, unidades)


def trabajador(cuenta, operaciones, semilla):
    rng = random.Random(semilla)
    for _ in range(operaciones):
        origen, destino = ("a", "b") if rng.random() < 0.5 else ("b", "a")
        cuenta.paso(origen, destino, rng.randint(1, UNIDAD_MAX))


def verificar(cuenta, operaciones_totales):
    fallos = []
    if cuenta.contador != operaciones_totales:
        fallos.append("contador")
    if cuenta.a + cuenta.b != cuenta.total:
        fallos.append("suma")
    if cuenta.limite_roto or cuenta.a < 0 or cuenta.b < 0:
```

```
        fallos.append("limite")
    return fallos


def ejecutar(hilos, operaciones, protegida, pausa_max=0.001, semilla=0):
    cuenta = Cuenta(protegida=protegida, pausa_max=pausa_max)
    ts = [threading.Thread(target=trabajador, args=(cuenta, operaciones, semilla * 100 + i), name=f"T
     {i + 1}")
          for i in range(hilos)]
    ini = time.perf_counter()
    for t in ts:
        t.start()
    for t in ts:
        t.join()
    duracion = time.perf_counter() - ini
    return cuenta, verificar(cuenta, hilos * operaciones), duracion
```

2.3. Plataforma y parámetros

Parámetro	Valor
Lenguaje / runtime	Python 3.12.3 (CPython, con GIL)
Núcleos lógicos disponibles	1 (contenedor de pruebas)
Threads por corrida	2 y 4
Pasos por thread	50
Pausa entre lectura y escritura	aleatoria uniforme en [0, 1] ms (`time.sleep`)
Repeticiones por escenario	100
Saldos iniciales / suma total	100 y 100 / 200

2.4. Un intercalado defectuoso concreto

Para tener evidencia reproducible, `intercalado.py` sustituye las pausas aleatorias por una `threading.Barrier(2)`: ambos threads se detienen en el mismo punto y luego continúan. El resultado es un intercalado concreto, no una coincidencia. Cada línea registra quién, qué, cuándo y sobre qué dato.

Listing 2: Traza sin protección (T1 y T2 transfieren 60 unidades de a hacia b)

```
protegida=False contador=1 a=-20 b=160 fallos=['contador', 'suma', 'limite']
    0.000 ms T1 lee contador 0
    0.058 ms T2 lee contador 0
    0.064 ms T2 escribe contador 1
    0.066 ms T2 lee a 100
    0.081 ms T1 escribe contador 1
    0.082 ms T1 lee a 100
    0.092 ms T2 escribe a 40
    0.092 ms T2 lee b 100
    0.099 ms T1 escribe a -20
    0.100 ms T1 lee b 100
    0.105 ms T2 escribe b 160
    0.139 ms T1 escribe b 160
protegida=True contador=2 a=40 b=160 fallos=[]
```

Resultado: `contador=1` (esperado 2), `a=-20` y `b=160` (suma 140, esperado 200). Se rompen los tres invariantes a la vez:

- **Actualización perdida del contador.** T1 y T2 leen 0 y ambos escriben 1.
- **Violación del límite (sobregiro).** T1 y T2 leen `a=100` y ambos aprueban la verificación `100 >= 60`; el segundo débito se aplica sobre el valor ya descontado y deja `a=-20`. Verificar y descontar son una sola decisión lógica y aquí quedaron separadas.
- **Pérdida de suma.** T1 y T2 leen `b=100` y ambos escriben 160: se debitaron 120 unidades de `a` pero `b` solo subió 60. Se perdieron 60 unidades y la suma quedó en $-20 + 160 = 140$.

2.5. Resultados de 100 repeticiones sin protección

Métrica (de 100 corridas)	2 threads	4 threads
Contador final incorrecto	100	100
Suma $a + b \neq 200$	100	100
Algún saldo negativo (límite roto)	82	99
Incrementos perdidos (media / máx.)	33.6 / 46 de 100	128.1 / 148 de 200
Desvío de la suma $\lvert a + b - 200 \rvert$ (media / máx.)	148.9 / 484	208.0 / 794

2.6. Análisis

- El fallo no es esporádico en esta configuración: aparece en las 100 corridas de ambos tamaños. Las pausas aleatorias entre la lectura y la escritura hacen que casi todos los pasos se solapen con los de otros threads.
- El efecto crece con el número de threads: con 4 threads se pierden 128 de 200 incrementos y el límite se rompe en 99 corridas.
- El GIL no evita el problema: protege la integridad del intérprete, no la atomicidad de una secuencia *leer–calcular–escribir* de varias sentencias, y `time.sleep` libera el GIL.
- Sin pausas el fallo también aparece (sección 3.4: 9 de 10 corridas con 2 threads y 10 de 10 con 4), solo que depende del cambio de contexto del planificador. Una ejecución sin fallo observado no demuestra ausencia de carrera.

3. Parte B — Demostrar la corrección

3.1. Invariantes (definidos antes de corregir)

Sea N el número total de pasos ejecutados (threads × pasos por thread).

1. **I1 (contador):** al terminar, `contador` = N.
2. **I2 (conservación):** en todo estado observable fuera de un paso, $a + b = 200$.
3. **I3 (límites):** en todo momento, $a \geq 0$ y $b \geq 0$.

I2 solo puede observarse fuera de un paso porque dentro de la transferencia existe un instante legítimo en que se debitó el origen y aún no se acreditó el destino. Por eso la unidad atómica debe ser el paso completo y no una asignación suelta.

3.2. Protección de la transición completa

Se usa un único `threading.Lock` que cubre el paso completo: lectura del saldo, verificación del límite, débito, crédito e incremento del contador. En el código, la diferencia entre ambas versiones es solo esta línea:

```
def paso(self, origen, destino, unidades):
    with self.lock:          # nullcontext() si protegida=False
        self.incrementar()
        self.transferir(origen, destino, unidades)
```

Un lock por variable no bastaría: la verificación `saldo >= u` lee `a`, y el débito escribe `a` y luego `b`; el invariante I2 relaciona ambas variables. Proteger solo la escritura tampoco serviría, porque la decisión depende de una lectura previa. Se mantienen las mismas pausas aleatorias dentro de la sección crítica a propósito, para comparar ambas versiones bajo idéntica carga.

3.3. Mismas pruebas, versión protegida

Métrica (de 100 corridas)	2 threads	4 threads
Contador final incorrecto	0	0
Suma $a + b \neq 200$	0	0
Algún saldo negativo	0	0

Con el intercalado forzado de la sección 2.4, el `Lock` impide que T2 entre mientras T1 está a mitad del paso. La barrera ya no puede reunir a ambos threads, vence su tiempo de espera y la ejecución resulta secuencial. T2 consulta `a=40`, ve que 40 <60 y rechaza la transferencia, de modo que el límite se respeta por diseño:

Listing 3: Traza con protección: T2 no entra hasta que T1 termina

```
  0.000 ms T1 lee contador 0
 50.178 ms T1 escribe contador 1
 50.183 ms T1 lee a 100
100.444 ms T1 escribe a 40
100.447 ms T1 lee b 100
```

```
150.640 ms T1 escribe b 160
150.791 ms T2 lee contador 1
200.994 ms T2 escribe contador 2
200.999 ms T2 lee a 40
```

Estado final: `contador=2`, `a=40`, `b=160`, suma 200. Los tres invariantes se cumplen. Las esperas de 50 ms de la traza son el tiempo de espera de la barrera, no un costo del lock.

3.4. Costo con y sin protección

Con pausas (50 pasos por thread, mediana de 100 corridas, tiempo total de la corrida):

Threads	Sin protección	Con Lock	Factor
2	0.0763 s	0.1647 s	2.2×
4	0.0488 s	0.3290 s	6.7×

Sin pausas (20 000 pasos por thread, mediana de 10 corridas):

Threads	Sin protección	Con Lock	Corridas con fallo (sin / con)
2	0.0586 s	0.0547 s	9 / 0
4	0.1091 s	0.1510 s	10 / 0

- Con pausas, el lock serializa los threads: la espera que antes se solapaba ahora ocurre dentro de la sección crítica. El tiempo con `Lock` crece de forma proporcional al número de threads (0.165 s a 0.329 s al duplicarlos). Es un costo del diseño de la prueba (una pausa dentro de la sección crítica), no del lock en sí.
- Sin pausas, el costo propio del lock es pequeño: con 2 threads la diferencia (0.0586 s frente a 0.0547 s) está dentro del ruido de medición; con 4 threads la versión protegida tarda cerca de 38 % más. Este costo incluye la contención por el lock y los cambios de contexto asociados.
- La versión sin protección es más rápida pero incorrecta: falló en 19 de las 20 corridas sin pausas. Una versión rápida que viola los invariantes no es una alternativa válida.

ADVERTENCIA METODOLÓGICA

Estas mediciones corresponden a un contenedor con un solo núcleo lógico y a una carga artificial con pausas. En una máquina con varios núcleos, la contención por el lock y los efectos de caché cambiarían las proporciones. Los tiempos sirven para comparar ambas versiones entre sí, no como cifras absolutas.

3.5. Atomicidad y visibilidad obtenidas

Atomicidad. El `Lock` garantiza exclusión mutua: mientras un thread está dentro de `paso()`, ningún otro puede ejecutar esa sección. Para el resto de los threads, el paso completo (contador, débito y crédito) aparece como una sola unidad indivisible; nadie observa el estado intermedio en que el origen ya fue debitado y el destino aún no fue acreditado. Esto restaura I2 e I3, porque la verificación del límite y el débito dejan de poder intercalarse.

Visibilidad. Cada `release()` de un lock establece una relación *happens-before* con el siguiente `acquire()` del mismo lock: todo lo que un thread escribió dentro de la sección crítica es visible para el siguiente que la adquiera. Además, `Thread.start()` y `Thread.join()` crean relaciones de orden con el thread principal, por lo que `verificar()` lee valores finales válidos tras el `join`. Happens-before no significa "ocurrió antes en el reloj", sino una garantía de orden observable.

3.6. Prueba automatizada

`test_invariantes.py` usa `unittest`. La prueba principal ejecuta 30 corridas con 2 y con 4 threads y falla si `verificar()` devuelve algún invariante roto. Las demás comprueban el verificador y los intercalados forzados.

Listing 4: test_invariantes.py

```python
import os
import unittest

from carrera import Cuenta, ejecutar, verificar
from intercalado import forzar


PROTEGIDA = os.environ.get("PROTEGIDA", "1") == "1"


class PruebasInvariantes(unittest.TestCase):
    def test_version_protegida_conserva_invariantes(self):
        for hilos in (2, 4):
            for r in range(30):
                c, fallos, _ = ejecutar(hilos, 30, PROTEGIDA, 0.0005, r)
                self.assertEqual(fallos, [], f"hilos={hilos} rep={r} estado={c.contador},{c.a},{c.b}"
    )

    def test_intercalado_forzado_con_proteccion(self):
        c, _ = forzar(True)
        self.assertEqual(verificar(c, 2), [])

    def test_verificador_detecta_estado_corrupto(self):
        c = Cuenta()
        c.contador, c.a, c.b, c.limite_roto = 1, -20, 230, True
        self.assertEqual(sorted(verificar(c, 2)), ["contador", "limite", "suma"])

    def test_intercalado_forzado_sin_proteccion_rompe_invariantes(self):
        c, _ = forzar(False)
        self.assertEqual(sorted(verificar(c, 2)), ["contador", "limite", "suma"])


if __name__ == "__main__":
    unittest.main()
```

Listing 5: Ejecución de las pruebas

```
$ python3 -m unittest -v
test_intercalado_forzado_con_proteccion ... ok
test_intercalado_forzado_sin_proteccion_rompe_invariantes ... ok
test_verificador_detecta_estado_corrupto ... ok
test_version_protegida_conserva_invariantes ... ok
Ran 4 tests in 5.269s
OK

$ PROTEGIDA=0 python3 -m unittest test_invariantes.PruebasInvariantes.
    test_version_protegida_conserva_invariantes
AssertionError: ['contador', 'suma', 'limite'] != [] : hilos=2 rep=0 estado=40,30,27
FAILED (failures=1)
```

La segunda ejecución quita la protección mediante la variable de entorno `PROTEGIDA=0` y la misma prueba falla de inmediato, lo que confirma que la prueba detecta la ruptura del invariante.

4. Conclusiones

1. Un incremento o una transferencia escrita en pocas líneas no es atómica: se descompone en leer, calcular y escribir, y su resultado depende del intercalado. Sin protección, los tres invariantes se rompieron en las 100 corridas con 2 y con 4 threads.
2. La unidad atómica debe cubrir la transición completa que mantiene el invariante: verificar el saldo, debitar, acreditar y contar. Proteger solo las escrituras o cada variable por separado no habría bastado.
3. Con un `Lock` sobre el paso completo se obtuvieron 0 violaciones en 400 corridas, y el `release/acquire` aporta además la visibilidad entre threads.
4. La corrección tiene costo: bajo contención y con trabajo dentro de la sección crítica, el tiempo creció hasta 6.7 veces. La sección crítica debe ser lo más corta posible.
5. Primero se demuestra la corrección y después se argumenta el rendimiento: la versión sin protección es más rápida, pero sus resultados son inválidos.
6. Una prueba automatizada que falle cuando el invariante se rompe protege la corrección ante cambios futuros del código.

Referencias bibliográficas

[1] A. Silberschatz, P. B. Galvin y G. Gagne, *Operating System Concepts*, 10.ª ed. Hoboken, NJ, USA: Wiley, 2018.

[2] A. S. Tanenbaum y H. Bos, *Modern Operating Systems*, 4.ª ed. Boston, MA, USA: Pearson, 2015.

[3] Python Software Foundation, "threading — Thread-based parallelism", *Python 3 Documentation*.

Disponible en: https://docs.python.org/3/library/threading.html

[4] Python Software Foundation, "unittest — Unit testing framework", *Python 3 Documentation*.

Disponible en: https://docs.python.org/3/library/unittest.html